Scenario 1: Bad Container Image Deployment

Objective

Trace an incomplete Kubernetes rollout from its first symptom to the Deployment image and Amazon ECR, then recover safely.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/bad-deploy.sh banking banking-backend banking-backend
```

The script changes the live Deployment image to a nonexistent tag. Give Kubernetes a minute to observe the failed rollout before troubleshooting. Do not run another scenario until this one has been recovered.

Situation

The banking application still serves users, but a new release has not completed.

Symptoms

- The rollout does not complete.
- New pods do not start while existing pods may remain healthy.

What You Know

- The issue began immediately after a banking backend release.
- The `banking-backend` Deployment has new pods that are not Ready.
- Existing pods may still serve traffic.

Start With the Symptom

1. Watch the rollout. It never finishes:

```
kubectl rollout status deployment/banking-backend -n banking --timeout=30s
```

1. List recent warning events in the namespace and look for `ErrImagePull`, `ImagePullBackOff`, `Failed to pull image`, or `does-not-exist`:

```
kubectl get events -n banking --field-selector type=Warning --sort-by=.lastTimestamp
```

Unavailable replicas can remain `0` in this scenario because the two old pods continue serving while Kubernetes creates one bad surge pod. Use the desired-versus-updated gap plus the image-pull event as the primary evidence.

Record the first event timestamp, failed pod, and requested image before digging deeper.

Troubleshooting

1. Check the Deployment summary.

```
kubectl get deployment banking-backend -n banking
```

Expected output: `READY` remains `2/2`, `AVAILABLE` remains `2`, but `UP-TO-DATE` is only `1`. This means the old replicas are healthy while the new rollout is incomplete.

1. Find the failed new pod.

```
kubectl get pods -n banking -l app=banking-backend
```

Expected output: two older pods are `Running`, while the newest pod is `ErrImagePull` or `ImagePullBackOff` and shows `0/1` Ready.

1. Inspect the failed pod and its recent events.

```
kubectl describe pod <pod-name> -n banking
```

Expected output: the **Events** section contains `Failed to pull image`, the requested tag ends in `does-not-exist`, and the registry reports that the image was not found.

1. Confirm the image configured on the Deployment.

```
kubectl get deployment banking-backend -n banking \
  -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
```

Expected output: an ECR image URL ending in `:does-not-exist`.

1. Compare the requested tag with the images that actually exist in ECR.

```
aws ecr describe-images --repository-name dev-cluster/banking-backend \
  --region us-east-1 --query 'imageDetails[].imageTags' --output table
```

Expected output: valid image tags are listed, but `does-not-exist` is absent. The missing ECR tag is the root cause.

Important Clues

Connect the replica mismatch with `ErrImagePull` or `ImagePullBackOff`, Kubernetes events, and the ECR image list.

Root Cause

Record your conclusion from the Kubernetes and ECR evidence before you apply the fix.

Fix

Roll back to the last working revision.

```
kubectl rollout history deployment/banking-backend -n banking
kubectl rollout undo deployment/banking-backend -n banking
```

Validation

```
kubectl rollout status deployment/banking-backend -n banking
kubectl get pods -n banking
```

Confirm the application still works and every replica is up to date and Ready.

DevOps Lesson

Validate an ECR tag before changing a Kubernetes Deployment. A healthy old ReplicaSet does not mean a release succeeded.

STAR Method

Situation

Summarize the failed rollout without naming the cause first.

Task

Explain the safe recovery objective.

Action

Describe the evidence path from the rollout status to Kubernetes events and ECR.

Result

State how the rollout recovery was verified.

Natural Spoken Version

Use the matching example in DevOps STAR Scenarios after completing the lab.

Brief STAR Example

One of our EKS releases referenced image tag `v2.4.1`, but only `v2.4.0` existed in ECR. My task was to restore a safe rollout without interrupting the old healthy replicas. I used the rollout status and Kubernetes events to confirm `ImagePullBackOff`, verified that the tag was missing in ECR, and rolled back the Deployment. The application remained available, the rollout recovered, and we added image-tag validation to the pipeline.

Scenario 2: Bad ConfigMap Rollout

Objective

Diagnose a rollout where application configuration and Kubernetes health probes no longer agree.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/break-config.sh student-portal
```

The script saves the current port, changes the ConfigMap, and restarts the student portal backend Deployment. Give Kubernetes a minute to observe the rollout failure before troubleshooting.

Situation

The student portal has reduced rollout capacity after a configuration release. Existing pods may still serve requests, but new pods do not become Ready.

Symptoms

- New pods fail to become Ready.
- Restart counts may increase while older pods still serve.

What You Know

- `student-portal-backend` has unavailable replicas and increasing container restarts.
- Kubernetes health events began after a rollout.
- The process in each new container appears to start.

Start With the Symptom

1. Compare Ready status and restarts on the old and new pods:

```
kubectl get pods -n student-portal -l app=student-portal-backend -w
```

1. List recent warning events and find the readiness-probe, liveness-probe, or BackOff event for the newest backend pod:

```
kubectl get events -n student-portal --field-selector type=Warning --sort-by=.lastTimestamp
```

Record the newest pod, first probe-event timestamp, and restart change.

Troubleshooting

1. Check the rollout and identify the unhealthy new pod.

```
kubectl get deployment student-portal-backend -n student-portal
kubectl get pods -n student-portal -l app=student-portal-backend
```

Expected output: the rollout is incomplete, and the newest pod is not Ready or is restarting while older pods remain Running.

1. Inspect the new pod.

```
kubectl describe pod <pod-name> -n student-portal
```

Expected output: events show readiness or liveness probe failures against port `4000`, often followed by `BackOff` or a container restart.

1. Read the application startup logs.

```
kubectl logs <pod-name> -n student-portal
```

Expected output: the application reports that it is listening on port `4001`. This conflicts with the probes checking port `4000`.

1. Compare the ConfigMap value with the Deployment probes.

```
kubectl get configmap student-portal-backend-config -n student-portal -o yaml
kubectl get deployment student-portal-backend -n student-portal \
  -o jsonpath='{.spec.template.spec.containers[0].readinessProbe.httpGet.port}{"\n"}'
```

Expected output: the ConfigMap contains `PORT: "4001"`, but the readiness probe returns `4000`. The port mismatch is the root cause.

Important Clues

Connect the probe failures to the application listening port and the port checked by the readiness and liveness probes.

Root Cause

Record the configuration and probe mismatch supported by your evidence before you apply the fix.

Fix

```
./scripts/chaos/break-config.sh student-portal --undo
```

The script restores the saved `PORT` value and restarts the Deployment.

Validation

```
kubectl rollout status deployment/student-portal-backend -n student-portal
kubectl get pods -n student-portal
```

Confirm every pod is Ready, restarts stop increasing, and the app works.

DevOps Lesson

Validate configuration and health-probe contracts together before rollout.

STAR Method

Situation

Summarize the partial-capacity rollout.

Task

Explain why only new pods needed investigation.

Action

Describe the event, log, ConfigMap, and probe comparison.

Result

State how readiness recovery was verified.

Natural Spoken Version

Use the matching example in DevOps STAR Scenarios after completing the lab.

Brief STAR Example

One of our internal applications stopped bringing new pods to Ready after a configuration rollout. My task was to identify why only the new pods failed. Kubernetes events and pod logs showed probe failures and an application listening on port 4001, while Kubernetes probes still checked port 4000. I restored the correct ConfigMap value and restarted the Deployment. All replicas became Ready, and configuration-to-probe validation was added to deployment checks.

Scenario 3: OOMKilled and Kubernetes Resource Limits

Objective

Use Kubernetes evidence to distinguish memory pressure from an unsafe container resource limit.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/shrink-limits.sh support-tickets
```

The script saves the current resources and applies a deliberately unsafe memory request and limit. Wait for restarts to appear before troubleshooting.

Situation

The support tickets service is unstable. Backend pods restart repeatedly while some requests still succeed.

Symptoms

- Backend pods restart repeatedly.
- Application stability degrades.

What You Know

- Container restart counts are increasing.
- The problem began after a workload resource change.

Start With the Symptom

1. Watch the backend pods. The newest one keeps restarting and ends up in `CrashLoopBackOff`:

```
kubectl get pods -n support-tickets -l app=support-tickets-backend -w
```

1. Check how much memory the healthy pods use:

```
kubectl top pods -n support-tickets
```

Record the restart timestamp, pod name, and the memory the healthy pods use. Confirm the exact `OOMKilled` reason with `kubectl describe` below.

Troubleshooting

1. Identify the restarting pod.

```
kubectl get pods -n support-tickets -l app=support-tickets-backend
```

Expected output: the older pods stay Running, while the newest pod's `RESTARTS` count is greater than zero and keeps increasing.

1. Inspect the previous container termination.

```
kubectl describe pod <pod-name> -n support-tickets
```

Expected output: under **Last State**, the reason is `OOMKilled` and the exit code is `137`.

1. Compare the memory a healthy pod uses with the configured limit.

```
kubectl top pods -n support-tickets
kubectl get deployment support-tickets-backend -n support-tickets \
  -o jsonpath='{.spec.template.spec.containers[0].resources}{"\n"}'
```

Expected output: healthy backend pods use about `16Mi`, but the Deployment now sets a `12Mi` memory limit. The new container exceeds that limit while starting, and Kubernetes terminates it.

1. Check the previous container logs for supporting evidence.

```
kubectl logs <pod-name> -n support-tickets --previous
```

Expected output: logs stop around the restart time and may not contain a graceful application error. `OOMKilled` in the pod state is the authoritative evidence.

Important Clues

Look for `Reason: OOMKilled`. Compare observed memory consumption with `resources.requests.memory` and `resources.limits.memory`.

Root Cause

Record the relationship between observed memory, the previous container state, and the configured limit before you apply the fix.

Fix

Restore the known baseline, then use observed utilization to propose a justified long-term request and limit.

```
./scripts/chaos/shrink-limits.sh support-tickets --undo
kubectl rollout status deployment/support-tickets-backend -n support-tickets
```

Do not remove the limit or choose an arbitrary large value.

Validation

Confirm pods remain Ready, restart counts stop increasing, memory stays below the limit, and requests succeed.

DevOps Lesson

Resource limits must reflect measured workload behavior. Admission control can accept a value that is operationally unsafe.

STAR Method

Situation

Summarize the unstable workload.

Task

Explain how you separated application, Kubernetes, and infrastructure causes.

Action

Describe the memory, restart, previous-state, and resource comparison.

Result

State how stability and memory headroom were verified.

Natural Spoken Version

Use the matching example in DevOps STAR Scenarios after completing the lab.

Brief STAR Example

An internal backend repeatedly restarted after its memory limit was reduced to 12 MiB. My task was to distinguish an application failure from a Kubernetes resource problem. I correlated the restarts and the

memory the healthy pods used with exit code 137 and `OOMKilled` in the pod state, then restored a measured memory limit. The restart loop stopped, pods remained Ready, and future resource values were based on observed usage.

Scenario 4: ALB and Kubernetes Ingress Routing Failure

Objective

Trace HTTP 404 responses across Route 53, ALB, Ingress, Service, endpoints, and pods.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/break-ingress.sh food-delivery
```

The script saves the current Ingress backend and changes the live route. Allow one or two minutes for the AWS Load Balancer Controller to reconcile before testing the public URL.

Confirm the failure from outside the cluster:

```
for i in {1..10}; do
  curl -s -o /dev/null -w '%{http_code}\n' \
    "https://food-delivery.$(cat .lab-domain)/"
done
```

Expected output: repeated HTTP `404` responses.

Situation

Users receive HTTP 404 from the food delivery home page even though the Kubernetes workloads report healthy.

Symptoms

- The public hostname returns HTTP 404.
- Kubernetes workloads remain Running and Ready.

What You Know

- `food-delivery.<lab-domain>` resolves and the ALB accepts connections.
- Pods are Running and Ready.

Start With the Symptom

1. Look at the body of the 404, not just the status code:

```
curl -s "https://food-delivery.$(cat .lab-domain)/"
```

Expected output: `Cannot GET /`. That is the Express backend's default 404 page, not the nginx frontend, which means public root traffic is reaching the backend instead of the frontend.

1. Compare it with an API path, which the backend is supposed to answer:

```
curl -s -o /dev/null -w '%{http_code}\n' "https://food-delivery.$(cat .lab-
domain)/api/restaurants"
```

Expected output: `200`. The ALB and backend work; only the route for `/` is wrong. Record the timestamp, then continue with Kubernetes and the AWS CLI.

Troubleshooting

1. Confirm that the workloads are healthy.

```
kubectl get pods -n food-delivery
```

Expected output: frontend and backend pods are Running and Ready. This moves the investigation away from pod health and toward request routing.

1. Inspect the public Ingress route.

```
kubectl describe ingress food-delivery -n food-delivery
```

Expected output: the `/` path points to `food-delivery-backend:4000`. It should point to `food-delivery-frontend:80`.

1. Confirm that both Services and endpoint sets exist.

```
kubectl get svc,endpoints -n food-delivery
```

Expected output: the frontend Service listens on port `80`, the backend listens on `4000`, and both have endpoints. This proves the failure is an incorrect Ingress destination rather than a missing Service or pod.

1. Confirm the ALB is present and serving the incorrect rule.

```
aws elbv2 describe-load-balancers --region us-east-1
aws elbv2 describe-listeners --load-balancer-arn <alb-arn> --region us-east-1
aws elbv2 describe-rules --listener-arn <listener-arn> --region us-east-1
```

Expected output: the ALB and HTTPS listener are active, and the host rule forwards traffic to the target group created from the incorrect backend Service. The Ingress backend selection is the root cause.

Important Clues

Follow DNS to ALB, listener, target group, Ingress, Service, endpoints, pod, and application. Determine why the backend answers a request intended for the frontend.

Root Cause

Record which routing hop sends traffic to the wrong destination before you apply the fix.

Fix

```
./scripts/chaos/break-ingress.sh food-delivery --undo
kubectl describe ingress food-delivery -n food-delivery
```

The source-of-truth manifest is `ingress/food-delivery-ingress.yaml`, whose backend is `food-delivery-frontend:80`.

Validation

Test `https://food-delivery.<lab-domain>/`, confirm the controller reconciles the Ingress, and verify that `/` returns `200` with the frontend page instead of `Cannot GET /`.

DevOps Lesson

Healthy pods do not guarantee healthy user traffic. Validate the whole routing path.

STAR Method

Situation

Summarize the user-visible 404 with healthy workloads.

Task

Explain the need to locate the failing routing hop.

Action

Describe the external, AWS, and Kubernetes traffic-path checks.

Result

State how external access was verified.

Natural Spoken Version

Use the matching example in DevOps STAR Scenarios after completing the lab.

Brief STAR Example

Users received HTTP 404 responses from one of our applications even though every pod was healthy. My task was to locate the failing hop in the public request path. The 404 body came from the Express backend rather than nginx, and the Ingress revealed that `/` targeted port 4000 instead of the frontend on port 80. I corrected the route, waited for ALB reconciliation, and confirmed successful external requests.

Scenario 5: High Application Latency

Objective

Measure latency from outside the cluster, isolate the slow workload, then investigate Kubernetes and apply a focused recovery.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/inject-latency.sh ecommerce 3000
```

The request reaches one backend pod because chaos state is stored per process. Only requests served by that pod are slow, so the symptom is intermittent. To affect every backend pod, use the port-forward method under Fix to send the same request to each pod.

Situation

Users report that ecommerce pages load slowly. Do not restart pods before scoping the problem.

Symptoms

- Ecommerce responses are intermittently slow.
- Pods remain Ready.

What You Know

- The issue affects `ecommerce-backend` while pods remain Ready.
- The start time does not match an AWS or database change.

Start With the Symptom

1. Time repeated API requests from outside the cluster:

```
for i in {1..10}; do
  curl -s -o /dev/null -w '%{http_code} %{time_total}s\n' \
    "https://ecommerce.$(cat .lab-domain)/api/products"
done
```

Expected output: every request returns `200`, but some take about three seconds while the rest are fast.

1. Time the static frontend the same way:

```
curl -s -o /dev/null -w '%{http_code} %{time_total}s\n' "https://ecommerce.$(cat .lab-domain)/"
```

Expected output: consistently fast. DNS, the ALB, and the frontend are fine, so the delay is in the backend.

Record the start time, the slow and fast response times, and how many requests were slow.

Troubleshooting

1. Confirm Kubernetes health.

```
kubectl get deployment,pods -n ecommerce
```

Expected output: the Deployment is fully available, pods are Ready, and restart counts remain stable. The application is slow but not unavailable.

1. Check resource usage.

```
kubectl top pods -n ecommerce
```

Expected output: CPU and memory remain near their normal baseline. This makes resource saturation and OOM restarts unlikely.

1. Time each backend pod directly to find the slow one.

```
kubectl get pods -n ecommerce -l app=ecommerce-backend
kubectl port-forward -n ecommerce pod/<pod-name> 4000:4000
# in a second terminal:
curl -s -o /dev/null -w '%{time_total}s\n' http://localhost:4000/api/products
```

Expected output: one pod answers in about three seconds and the others answer quickly. The slowdown lives inside one process, not in the network path.

1. Check that pod's chaos state through the same port-forward.

```
curl -s http://localhost:4000/api/chaos
```

Expected output: JSON containing `"latencyMs":3000`. This in-memory setting explains requests taking about three seconds while Kubernetes remains healthy.

Important Clues

Determine why one or more backend containers add time before every response. Use per-pod response times and the chaos-state endpoint as evidence.

Root Cause

Record the workload behavior that accounts for the response time while Kubernetes resource metrics remain normal, before you apply the fix.

Fix

Clear the injected latency. Chaos state is per process, so reset every backend pod. The simplest way is through a port-forward to each pod:

```
kubectl port-forward -n ecommerce pod/<pod-name> 4000:4000
# in a second terminal:
curl -s -X POST http://localhost:4000/api/chaos/reset
```

`./scripts/chaos/reset.sh ecommerce` does the same through the public URL, but it only reaches whichever pod the load balancer picks, so repeat it and re-check each pod.

Validation

Repeat the timing loop from Start With the Symptom. Confirm every request is fast again, every pod reports `"latencyMs":0`, and pods remain healthy.

DevOps Lesson

Measure and isolate the slow workload before taking action. A restart may hide evidence without explaining the cause.

STAR Method

Situation

Summarize the latency report and initial scope.

Task

Explain the need to identify the affected workload before acting.

Action

Describe the external timing, per-pod timing, Kubernetes, and workload-state evidence.

Result

State how normal latency and healthy pods were verified.

Natural Spoken Version

Use the matching example in DevOps STAR Scenarios after completing the lab.

Brief STAR Example

Users reported slow responses from one of our applications while its EKS pods remained healthy. My task was to isolate the source before restarting anything. Timing requests showed API calls taking about three seconds on one backend pod, while CPU, memory, readiness, and restarts stayed normal. I found and cleared a 3000 ms application delay setting on that pod, after which response times returned to baseline.